

Intro to Linux and the Command Line

Dr Jake Mac Uilliam

Introduction

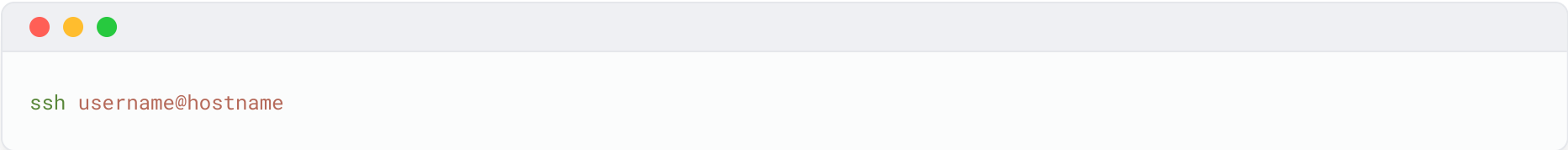
- Linux is an open-source operating system widely used in
 - Servers,
 - HPC systems,
 - Cloud computing,
 - Research environments ,
 - Mobile devices through Android.
- The first release of Linux was 17th September 1991 and was developed by Linus Torvalds.
- Interactions with the Linux operating system usually happen through the command line, though graphical interfaces and desktop environments (GNOME, KDE Plasma, etc) do exist.

Linux Interface: Terminals

- The terminal is how we will interact with our machine.
- It is keyboard based, and has many shortcuts.
- Available in different forms on different operating systems
 - Linux: Terminal, Konsole, xterm, gnome-terminal ...
 - macOS: Terminal, iTerm2 ...
 - Windows: Command Prompt, PowerShell, Windows Terminal ...

SSH

- SSH(Secure SHell) is a protocol for accessing machine remotely.
- To connect to these machine we use SSH keys.
- To connect to a remote machine we use

A terminal window with a light gray title bar containing three colored window control buttons (red, yellow, green). The main area is white and contains the text 'ssh username@hostname' in a monospaced font. 'ssh' is green, and 'username@hostname' is red.

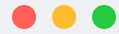
```
ssh username@hostname
```

- Here username is your username account on the remote machine, and hostname is the address of the machine.
- SSH can be blocked on some networks. In UCD, make sure you are connected to Eduroam.

The Linux Shell

- The shell is the command-line interface for interacting with the operating system.
- The shell allows us to create and move files, create directories, run programs, and much more.
- Bash (Bourne Again SHell) is the shell we will cover in this course as it is the most common shell on Linux. Alternatives include zsh, fish, and many more.

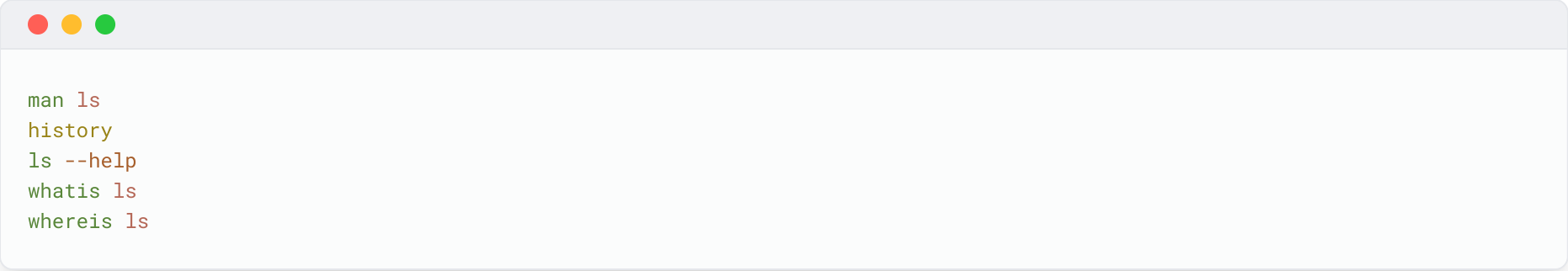
Basic Commands



```
pwd          # Prints the current working directory
ls           # Lists files in the current directory
touch file   # Creates a new empty file called file
cd dir       # Changes to the directory called dir
mkdir dir    # Creates a new directory called dir
cp source dest # Copies a file from source to dest
mv source dest # Moves a file from source to dest
rm file      # Removes a file called file
echo $VAR    # Prints the value of the variable VAR
man command  # Displays the manual page for command
```

Getting Help

- `man` displays the user manual (man pages) of any command, including optional flags. Use `q` to quit the man page
- `history` displays the last 500 commands you entered
- `help` for quick help
- `whatis` gives a one-line description
- `whereis` gives the location of the command on the machine



```
man ls
history
ls --help
whatis ls
whereis ls
```

Options

- Many commands can take options to modify their behavior.
- For example, `ls -a` will list files including hidden files, and `ls -t` will sort files by modification time.
- We can combine options, so `ls -at` will list all files sorted by modification time.
- The man pages (manual pages) list all options available for a command.

Wildcards

- Consider a directory with the following files: `001.txt`, `002.txt`, ... `other.dat`.
- We are going to use the "wildcards" `*`, `?`, `[]` to match files.
 - `*` matches one or more occurrences of any character. `ls *.txt` will list all files ending with `.txt`.
 - `?` matches exactly one occurrence of any character. `ls 00?.txt` will list files like `001.txt` and `002.txt`.
 - `[]` matches any one character inside the brackets. `ls 00[1-3].txt` will list `001.txt`, `002.txt`, and `003.txt`, but not `004.txt`.

Seeing File Content

- To see the content of a file, we can use commands like `cat`, `less`, or `more`.
- For example, `cat filename` will display the content of the file named "filename".
- The command `less filename` allows you to scroll through the file content.
- Use `head filename` to see the first few lines and `tail filename` to see the last few lines of a file.
- These all have options, so check the man pages for more details.

Practical: Basics

```
home/
├── dir01/
│   └── subdir/
├── dir02/
│   └── sub02a/
│       ├── sub02a1/
│       │   ├── nest02a/
│       │   └── nest02b/
└── dir03/
    ├── sub03a/
    │   └── sub03a1/
    ├── sub03b/
    │   ├── sub03b1/
    │   └── nested/
```

1. Create the above directory structure
2. Print the following message to the screen:

```
This is a simple message.
```

```
It can be done over multiple lines.
```

3. Repeat 2, but write it to a file called `correct.txt` instead

Practical: Basics

4. Write the following to a file called `incorrect.txt`

```
This is not a correct meesage.
```

```
It can't be done over multiple llnes.
```

```
It can't be done over multiple llnes.
```

5. Compare the two files using `wc` and `diff`.

6. Create files `01.c`, `02.c`, `03.c`, `04.c`, `05.c`, `06.c`, `17.c`, `18.c`, `19.c`, `20.c`, `21.c`, `32.c`, `33.c` inside `dir01`. Use wildcards to;

(a) Move any even numbered files which start with `"0"` to `dir02`.

(b) List files ending in `l`.

(c) Copy any odd numbered files back one directory.

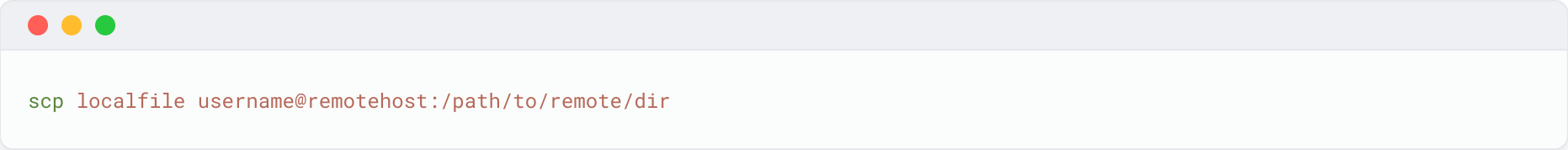
(d) Remove all remaining files except `01.c`.

Editing Files

- Editing files over SSH isn't as easy as opening Notepad or Word.
- Instead, we must use text editors that run in the terminal.
- The three most common editors are:
 - `nano`,
 - `vim`,
 - `emacs`.
- `nano` is the most beginner-friendly, while `vim` and `emacs` are more powerful but have steeper learning curves.
- The syntax for opening a file in an editor is the same for all three: `editor filename`.

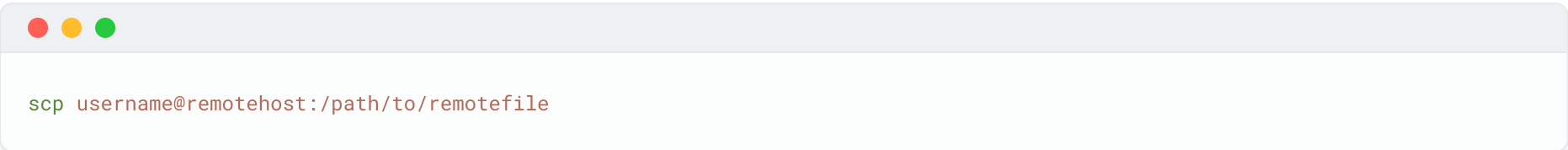
Transferring Files

- Sometimes, we need to transfer files between our local machine and the remote server.
- The most common command for transferring files is `scp` (Secure Copy Protocol).
- The syntax for copying a file from your local machine to the remote server is:



```
scp localfile username@remotehost:/path/to/remote/dir
```

- To copy a file from the remote server to your local machine, use:



```
scp username@remotehost:/path/to/remotefile
```

- Alternatively, you can use `sftp username@remotehost` which opens an interactive file transfer session. `put filename` moves `filename` from your machine to the remote machine and `get filename` moves `filename` from the remote machine to your machine. Commands such as `ls`, `cd`, and `mv` work here too.

Shell Scripts

- We can automate and combine commands in a file called a (shell) script.
- A shell script is a text file containing a series of commands that the shell can execute.
- They are very useful for automating repetitive tasks, running multiple commands in sequence, or setting up environments.
- The convention is that the file extension for shell scripts is `.sh`.
- A sample shell script is given on the next slide.

Sample Shell Script

SCRIPT.SH

```
1  #!/bin/bash
2  for I in {1..10}
3  do
4      echo "This is Hello World number $I"
5      if [ $I -eq 5 ]; then
6          echo "We're halfway through the loop!"
7      fi
8  done
```

Running a Shell Script

- To run a shell script, we need to make it executable first.
- Use the command `chmod +x script.sh` to make the script executable.
- Then, we can run the script with `./script.sh`.
- Let's go through the script line by line to understand what it does.

What Does the Script Do?

- `#!/bin/bash` - This is called the shebang. It tells the system where to find the shell.
- `for I in {1..10}` - This starts a loop that will iterate from 1 to 10. Note that it is convention to use uppercase for variables.
- `do` - This indicates the start of the loop body.
- `echo "This is Hello World number $I"` - This prints a message to the terminal, where `$I` is replaced by the current value of `I`.
- `if [ $I -eq 5 ]; then` - This checks if `I` is equal to 5.
- `echo "We're halfway through the loop!"` - If `I` is 5, this message is printed.
- `done` - This indicates the end of the loop body.

Important Points

- Variables are typically uppercase.
- For loops require a `do` and `done` to define the loop body.
- If statements require a `then` and end with `fi`.
- If we want to access the value of a variable, we need `$`.
- Checking equality requires `-eq` for integers.
- Shell scripts can also include `while` loops, `case` statements, and functions for more complex logic.
- BASH IS WHITESPACE SENSITIVE! Often errors arise due to incorrect indentation or additional whitespace.
- Comments can be added using `#` at the start of a line.

Permissions

- Every file and directory has permissions that control who can read, write, or execute it.
- The `chmod` command is used to change file permissions.
- The syntax is `chmod [ugo] [+ -] [rwx] filename`
- `u` is for user, `g` is for group, and `o` is for others.
- `+` adds a permission, `-` removes a permission.
- `r` is read, `w` is write, and `x` is execute.

Practical: Editing Files and Scripting

1. Edit your `.bashrc` file. If it does not exist, create a new one. Open your editor `$HOME/.bashrc`.

- Add the line `alias rm="rm -i"`
- Save and quit the `.bashrc` file.
- Use `source ~/.bashrc` to reload your `.bashrc`.
- Create a new file, `test.xxx`, and then delete it.

2. Create a `for` loop which does the following;

- for a variable `x` in the sequence of $0 \rightarrow 10$, create a new file with a `.txt` extension.
- if the number is divisible by 2, change the file extension to `.py`
- for any remaining text files divisible by 3, change the extension to a `.sh` file, and write the line `Hello from file x`, with `x` referring to the file name
- run the `.sh` files. The output upon running your main script should be:

```
Hello from file03  
Hello from file09
```

Practical: Editing Files and Scripting

3. Write a bash script to;

- Check and see if a an executable file of your choosing exists
- If it does, display `file_path` exists, where `file_path` is the path to your chosen file.
- Check to see if you can write to the file. If so, display `"You have permissions to edit file_path"`. If you cannot, display `"You do not have permissions to edit file_path. Contact the file owner to edit the file permissions"`. Disable your write abilities to that file and check that it works.

grep

- The `grep` command is used to search for text in files.
- The basic syntax is `grep pattern filename`.
- For example, `grep "Hello" script.sh` will search for the word `"Hello"` in the file `script.sh`.
- If you want to search all files and subdirectories for the phrase `"Hello World"`, you can use: `grep -r 'Hello World'`.

The Pipe Operator

- The pipe operator `|` allows us to send the output of one command as input to another command.
- For example, we can use `grep` to search for a pattern in a file and then use `wc -l` to count the number of lines that match:

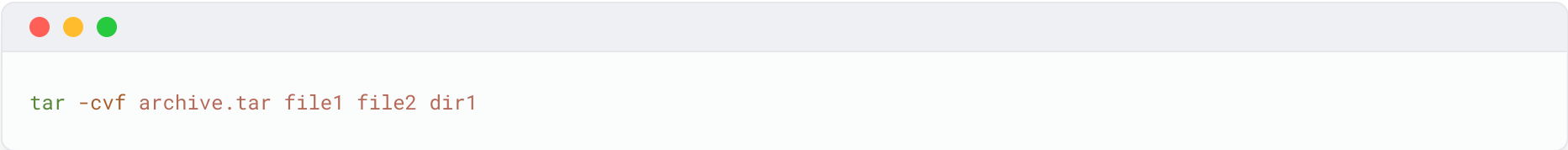


```
grep "pattern" file.txt | wc -l
```

- This will count how many lines in `file.txt` contain the pattern `pattern`

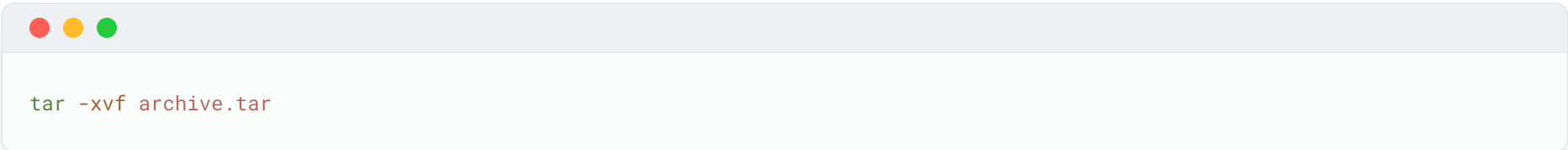
Archives

- It is very useful to be able to group files and directories together in a compressed format.
- On Linux, we can use the `tar` command to create and extract archives.
- To create an archive, we use:



```
tar -cvf archive.tar file1 file2 dir1
```

- To extract an archive, we use:



```
tar -xvf archive.tar
```

- The `-c` option creates an archive, `-x` extracts it, `-v` is for verbose output, and `-f` specifies the filename.

Practical: Archives, grep, and Piping

```
This is line 1 of test(01/02).  
This will also be used for string searching.  
This is line 3.
```

1. Make a new directory, `zipped`, and within it, create two files, `test01.txt` and `test02.txt` with the above message in respective files.
 - Make a copy of the directory, and create `tar` archives from both of them.
 - Create a new file `xxx` and use the `-r` flag to append it to the copied archive.
 - Zip both archive files using either `gzip` or the `-t` flag to view the contents of both archive files.
 - Usually with `tar` archives, we want to make them viewable and sometimes writable for others. Check the permissions and add writing privileges to `group` and `other`.
 - Unzip the copied archive and extract all data.

Practical: Archives, grep, and Piping

2. Use varieties of the `grep` command to search for the following in `National_Service_proj.txt` ;
 - Every occurrence of the word `Class`
 - Every occurrence of the word `resources` , case insensitive, showing 7 lines either side of the word.
 - Highlight every occurrence of `Class C` , highlighting the colour and showing the line number.
 - Use the `-l` option to search for files which contain the string `line` .

Practical: Archives, grep, and Piping

3. Use `echo` to write the following lines to a file, `lines.txt`. `This is line 1.` `This is line 2.` `This is line 3.` `This is line 4.`

- Use `cat` or `less` to view the contents of the file.
- Delete the file created above, then repeat but use `( | )` to directly pipe the output to the screen.
Check your directory, there should be no additional output file.

4. Use `grep` to search for the string `'class'` in `National_Service_proj.txt` and pipe it to `less`.

- Repeat the above but use a second pipe to `grep` for `'CPU'`.
- Pipes can be used as an `OR` statement for `grep` when specifying strings in the format `str1\|str2`.
Use this to repeat the above, but use the `OR` statement to search for `'CPU'`, `'Storage'`, `'Duration'` and `'Proposal'`.